pqc-scanner

Step-by-Step Test Guide

This guide walks you through testing the Enigma pqc-scanner on a sample multi-language project. You will scan Python, Java, Go, JavaScript, and nginx config files for quantum-vulnerable cryptography.

Generated: 2026-03-22

PREREQUISITES

You need:

- Python 3.10+ installed

- The Enigma project at C:\Users\jbro1\Desktop\Enigma

- pip install cryptography (for certificate scanning)

The demo project is already created at:

```
demo/sample_vulnerable_project/
```

It contains intentionally vulnerable code across 6 files and 4 languages.

STEP 1: Run a Basic Text Scan

Step 1.1 - Open a terminal in the Enigma directory

```
cd C:\Users\jbro1\Desktop\Enigma
```

Step 1.2 - Run the scanner on the demo project

```
python phases/phase6_scanner/pqc_scanner.py scan demo/sample_vulnerable_project
```

Expected output:

- Files Scanned: 6
- Total Findings: 29
- Risk Score: 100/100
- Severity: 16 CRITICAL, 9 HIGH, 4 MEDIUM

What to look for:

Each finding shows the file, line number, code snippet, what's wrong, and what to replace it with. CRITICAL means Shor's algorithm breaks it. HIGH means deprecated or broken. MEDIUM means Grover-weakened.

Exit codes:

- Exit 0: No findings (quantum-safe!)
- Exit 1: Findings found (non-critical)
- Exit 2: CRITICAL findings found

STEP 2: Generate an HTML Report

Step 2.1 - Generate the visual report

```
python phases/phase6_scanner/pqc_scanner.py scan demo/sample_vulnerable_project --format html
--output demo/scan_report.html
```

Step 2.2 - Open it in your browser

```
start demo\scan_report.html
```

What you'll see:

A dark-themed dashboard with a large risk score (100), total findings count, files scanned, and a color-coded table of every finding. CRITICAL findings are red, HIGH are orange, MEDIUM are yellow. Each row shows the algorithm, code snippet, file location, and PQC replacement.

STEP 3: Generate SARIF for CI/CD

SARIF (Static Analysis Results Interchange Format) is the standard way to feed scanner results into GitHub Code Scanning and Azure DevOps.

Step 3.1 - Generate SARIF output

```
python phases/phase6_scanner/pqc_scanner.py scan demo/sample_vulnerable_project --format sarif --output demo/scan_report.sarif
```

Step 3.2 - Inspect the SARIF file

Open demo/scan_report.sarif in any text editor. You'll see JSON with:

- version: '2.1.0' (SARIF standard version)

- runs[0].tool.driver.rules[] (17 quantum-vuln rules)

- runs[0].results[] (29 findings with locations)

Step 3.3 - Use in GitHub Actions (example workflow)

```
name: PQC Scan on: push jobs: scan: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - run: python pqc_scanner.py scan . --format sarif -o results.sarif - uses: github/codeql-action/upload-sarif@v3 with: sarif_file: results.sarif
```

STEP 4: Scan Certificates

The scanner can also analyze X.509 certificates for quantum-vulnerable algorithms.

Step 4.1 - Scan a live TLS endpoint

```
python phases/phase6_scanner/pqc_scanner.py scan-tls google.com
```

Expected:

> [CRITICAL] ECDSA-256: CN=*.google.com ECDSA P-256 certificate -- broken by Shor's algorithm Fix: Replace with ML-DSA certificate (FIPS 204)

Step 4.2 - Scan certificate files

```
python phases/phase6_scanner/pqc_scanner.py scan-certs /path/to/certs/
```

Scans all .pem, .crt, .cer, .der files in the directory recursively.

STEP 5: Scan Your Own Code

Now try it on a real project. Point the scanner at any codebase:

Step 5.1 - Scan any directory

```
python phases/phase6_scanner/pqc_scanner.py scan C:\path\to\your\project
```

Step 5.2 - Filter by severity

```
python phases/phase6_scanner/pqc_scanner.py scan . --severity critical
```

Only shows CRITICAL findings (Shor-vulnerable key exchange).

Step 5.3 - Try scanning Enigma itself

```
python phases/phase6_scanner/pqc_scanner.py scan phases/phase2_classical
```

This scans our own crypto code. Expected: 150+ findings (we built RSA, AES, ECDSA etc. from scratch, so the scanner correctly flags them all).

INTERPRETING RESULTS

Severity	Meaning	Action	Timeline
CRITICAL	Broken by Shor (RSA, ECDH, DH)	Replace with ML-KEM/ML-DSA	IMMEDIATE
HIGH	Deprecated or broken (3DES, MD5, RC4)	Upgrade algorithm	1-3 months
MEDIUM	Grover-weakened (AES-128, SHA-1)	Upgrade key size	3-6 months
LOW	Suboptimal but not broken	Review and plan	6-12 months
INFO	Informational (crypto lib detected)	Audit usage	Ongoing

Risk Score Interpretation:

0-20: Low exposure (mostly quantum-safe)

21-50: Moderate (some key exchange needs migration)

51-80: High (significant quantum-vulnerable surface)

81-100: Critical (urgent migration needed)

PQC Replacement Quick Reference:

Vulnerable	Replace With	NIST Standard
RSA (key exchange)	ML-KEM-768	FIPS 203
RSA (signatures)	ML-DSA-65	FIPS 204
ECDH / DH	X25519 + ML-KEM-768 hybrid	FIPS 203
ECDSA / Ed25519	ML-DSA-44 or ML-DSA-65	FIPS 204
AES-128	AES-256	Same standard, bigger key
SHA-1 / MD5	SHA-256 or SHA-3	FIPS 180-4 / FIPS 202